ESP32-CAM 스테레오 비전 거리 측정

프로젝트 정리 문서 (진행 경과 · README · 코드 설명 · 측정 결과 · 업로드 기록)

저장소	github.com/Lumiela/helpai (브랜치 `Height`)
작성일	2026-11-10

1부. 진행 경과

출처: 진행 보고서「높이 측정 모델 진행 - 이지후」(2026-10-16 ~ 2026-11-10)

목표와 접근 방식

미니 스마트팜에서 카메라 2대로 식물까지의 거리를 재고, 이를 높이 측정에 활용하는 것이 목표입니다. 두 카메라에 같은 물체가 가로로 어긋나 보이는 정도(시차 d)로 거리를 계산합니다.

```
Z = f × B / d      (f: 초점거리 px, B: 카메라 간격, d: 시차 px)
예) f = 700px, B = 10cm, d = 35px  →  Z = 700 × 0.1 / 35 = 2m
```

식물 검출과 결합하는 구상 (YOLO)

```
Camera L → YOLO → 식물 Bounding Box → 중심점 (xL, yL)
Camera R → YOLO → 식물 Bounding Box → 중심점 (xR, yR)
    → disparity = xL - xR → Z = fB / disparity → 식물까지 거리
```

카메라 2대와 YOLO만으로 거리가 정확히 나오지는 않으며, 다음 과정이 필요합니다.

1. 두 카메라의 위치 고정
2. 카메라 캘리브레이션
3. 렌즈 왜곡 보정
4. Stereo Rectification (평행화)
5. 좌/우 이미지에서 동일한 물체 대응점 찾기
6. Disparity 계산
7. 삼각측량으로 깊이 계산

특히 카메라 캘리브레이션과 두 카메라의 촬영 동기화가 중요합니다.

방식	결과	난이도
단일 카메라 + YOLO	식물 위치/크기	낮음
2카메라 + YOLO + Stereo	식물까지 거리	중간
2카메라 + Stereo Matching	전체 Depth Map	중~상
2카메라 + YOLO + Depth Map	식물별 3D 위치	중~상

미니 스마트팜에서는 **좌/우 카메라 → YOLO 식물 검출 → 좌우 동일 식물 매칭 → disparity → 거리 계산** 구조가 가장 현실적입니다. 현재 코드는 이 중 스테레오 매칭으로 전체 Depth Map을 만드는 단계까지 구현되어 있습니다.

버전별 진행

버전	날짜	내용	주요 설정
v1	2026-10-16	카메라 2대 스테레오 방식으로 거리 맵 생성	간격 50mm, 화각 72°, `BLOCK_SIZE` 9, `MAX_DIST_CM` 200
v2	2026-11-10	v1의 설정 요소 변경 및 시차 보정값 추가 → 거리 정확도 증가	간격 50mm, 화각 72°, `BLOCK_SIZE` 5, `MAX_DIST_CM` 500, `DISP_OFFSET_PX` 4.1

버전	날짜	내용	주요 설정
v3	2026-11-10	v2의 한계(맨 왼쪽 부분 거리 측정 불가) 해결	간격 50mm, 화각 72°, `BLOCK_SIZE` 5, `MAX_DIST_CM` 500, `DISP_OFFSET_PX` 4.1

실측 거리와 비교

같은 장면(`사용한 파일/`의 `CL.jpg`, `CR.jpg`)에서 가장 가까운 지점과 가장 먼 지점을 실측 거리와 비교한 결과입니다.

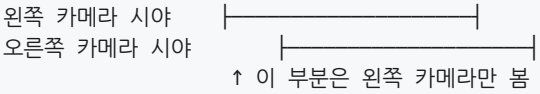
지점	실측 거리	v1	v2	v3
가장 가까운 지점 (min)	35 cm	44 ~ 47 cm	40 ~ 43 cm	40 ~ 43 cm
가장 먼 지점 (max)	65 cm	89 ~ 91 cm	62 ~ 78 cm	61 ~ 78 cm
오차 (가까운 지점)		+26 ~ +34%	+14 ~ +23%	+14 ~ +23%
오차 (먼 지점)		+37 ~ +40%	-5 ~ +20%	-6 ~ +20%

오차 행은 위 측정값과 실측값으로 계산한 값입니다. v1은 모든 거리가 실제보다 크게 나오는 **심한 오차**가 있었고, v2에서 시차 보정값을 넣은 뒤 가까운 곳과 먼 곳의 오차가 모두 줄었습니다. v3은 v2와 정확도가 비슷하면서 왼쪽 가장자리까지 측정 범위가 넓어졌습니다.

한계: 맨 왼쪽 부분의 거리가 측정되지 않음 (v2)

원인 1. 물리적 한계 — 오른쪽 카메라에 안 보이는 영역

두 카메라는 5cm 떨어져 있어 보는 범위가 조금 다릅니다. 왼쪽 사진의 맨 왼쪽 띠는 오른쪽 사진에 찍히지 않아 짝을 찾을 수 없으므로, 어떤 코드로도 거리를 계산할 수 없습니다. 이 띠의 폭은 시차와 같아서 현재 설정에서 가까운 물체(35cm)는 약 50px, 먼 물체(65cm)는 약 25px입니다.



원인 2. 계산 방식의 한계 — v3에서 해결

OpenCV의 StereoSGBM은 픽셀마다 오른쪽 사진에서 왼쪽 방향으로 탐색 범위만큼 짝을 찾습니다. 왼쪽 가장자리에서는 탐색 범위가 사진 밖으로 나가므로, OpenCV가 왼쪽 탐색 범위만큼의 픽셀을 통째로 측정 불가로 처리합니다. 이 장면에서는 탐색 범위가 약 100px이라 왼쪽 101px가 비어 있었고, 그중 절반 이상은 실제로 측정할 수 있는 영역이었습니다.

v3의 해결 방법: 두 사진 왼쪽에 탐색 범위만큼 검은 여백을 붙여 계산한 뒤 잘라냅니다. 원인 1에 해당하는 픽셀(`x - 시차 < 0`, 가장 먼 배경의 시차보다 왼쪽인 열)만 제외합니다. 그 결과 유효 픽셀 비율이 84%에서 95%로 늘었고, 측정 가능한 가로 범위가 30 ~ 639px가 되었습니다 (자세한 내용은 3부 5-3).

2부. README — 설치와 실행

폴더 구성

```
stereo/
├─ esp32_stereo_cam/esp32_stereo_cam.ino   # ESP32-CAM 펌웨어 (두 보드 공용)
├─ stereo.py                               # PC용 캘리브레이션 + 거리 측정
├─ stereo_colab.ipynb                      # Colab용 최신본: 캘리브레이션 + 사진 2장 → 거리 맵 (v3와 코드 같음, 기본 화각 81°)
├─ stereo_colab_v1.ipynb                   # Colab v1: 사진 2장 → 거리 맵 (캘리브레이션 셀 없음), 실행 결과 포함
├─ stereo_colab_v2.ipynb                   # Colab v2: 캘리브레이션 셀 복원 + 시차 보정값(DISP_OFFSET_PX), 실행 결과 포함
├─ stereo_colab_v3.ipynb                   # Colab v3: v2 + 왼쪽 가장자리까지 시차 계산, 실행 결과 포함
├─ 사용한 파일/                            # 측정에 사용한 이미지와 npy 파일
│   ├─ CL.jpg, CR.jpg                      #   사용한 왼쪽, 오른쪽 이미지
│   ├─ depth_cm (HFOV_DEG 72.0, 근사)_v1.npy  #   이 이미지로 계산한 거리 맵 (cm, 화각 72°, 현재 v3 실행 결과와 같은 값)
│   └─ depth_cm(HFOV_DEG 78.0, 근사).npy     #   이 이미지로 계산한 거리 맵 (cm, 화각 78°로 실행했던 이전 결과)
├─ 코드_설명.md                            # 코드 상세 설명서
├─ ESP32-CAM_스테레오_거리측정_정리.pdf     # 진행 경과 + README + 코드 설명 + 측정 결과를 묶은 PDF
├─ requirements.txt
└─ README.md
```

준비물

- ESP32-CAM (AI-Thinker, OV2640) 2대 + USB-TTL 업로더
- 두 카메라를 **평행하게 고정**할 판 (간격 6~10cm 권장, 측정 중 절대 움직이면 안 됨)
- 체커보드 (예: 10x7칸, 한 칸 25mm를 A4에 인쇄해 평평한 판에 부착)
- PC와 ESP32가 같은 2.4GHz Wi-Fi에 연결되어 있어야 함

1. 펌웨어 업로드

1. Arduino IDE → 보드 매니저에서 `esp32` 설치 → 보드: **AI Thinker ESP32-CAM**
2. `esp32_stereo_cam.ino`에서 `ssid`, `pass`, IP 대역(공유기에 맞게) 수정
3. 왼쪽 보드: `#define CAM_SIDE 0` 으로 업로드 → 192.168.0.101
4. 오른쪽 보드: `#define CAM_SIDE 1` 으로 업로드 → 192.168.0.102
5. 브라우저에서 `http://192.168.0.101/capture` 접속해 사진이 나오면 성공

> 왼쪽/오른쪽은 **카메라 뒤에서 피사체를 바라볼 때** 기준입니다. 바뀌면 거리가 측정되지 않습니다.

2. PC 설정

```
pip install -r requirements.txt
```

`stereo.py` 상단의 `BOARD`, `SQUARE_MM`을 실제 체커보드에 맞게 수정하세요.

3. 실행 순서

단계	명령	설명
0	`python stereo.py test`	두 카메라 영상/FPS 확인
1	`python stereo.py collect`	체커보드를 여러 위치·각도로 SPACE 눌러 20~30쌍 저장
2	`python stereo.py calibrate`	`stereo_calib.npz` 생성, RMS 오차 확인 (1.0px 미만 목표)
3	`python stereo.py measure`	거리 측정

measure 조작

- **왼쪽 클릭**: 해당 지점 거리 측정 (cm 표시)
- **오른쪽 클릭**: 화면 중앙으로 복귀
- **s**: 스크린샷 저장 / **q**: 종료

Colab 노트북 실행 순서

노트북을 Colab에서 열고 셀을 위에서부터 순서대로 실행합니다.

순서	`stereo_colab.ipynb`	`stereo_colab_v2.ipynb`	`stereo_colab_v3.ipynb`	`stereo_colab_v1.ipynb`
1	셀 0: 준비 (라이브러리, 한글 폰트)	셀 0: 준비 (라이브러리, 한글 폰트)	셀 0: 준비 (라이브러리, 한글 폰트)	셀 1: 설정값 입력 (기본 화각 72°)
2	셀 1: 설정값 입력 (기본 화각 81°)	셀 1: 설정값 입력 (기본 화각 72°)	셀 1: 설정값 입력 (기본 화각 72°)	셀 2: 왼쪽, 오른쪽 사진 업로드 (+ 선택: `stereo_calib.npz`)
3	셀 C1~C3 **(선택)**: 캘리브레이션	셀 C1~C3 **(선택)**: 캘리브레이션	셀 C1~C3 **(선택)**: 캘리브레이션	셀 3~5: 정렬 → 시차 계산 → 거리 계산
4	셀 2: 왼쪽, 오른쪽 사진 업로드	셀 2: 왼쪽, 오른쪽 사진 업로드	셀 2: 왼쪽, 오른쪽 사진 업로드	셀 6: 결과 보기
5	셀 3~5: 정렬 → 시차 계산 (왼쪽 가장자리 포함) → 거리 계산	셀 3~5: 정렬 → 시차 계산 → 거리 계산	셀 3~5: 정렬 → 시차 계산 (왼쪽 가장자리 포함) → 거리 계산	셀 7: 원하는 지점 거리 측정
6	셀 6: 결과 보기	셀 6: 결과 보기	셀 6: 결과 보기	셀 8 **(선택)**: 결과 다운로드
7	셀 7: 원하는 지점 거리 측정	셀 7: 원하는 지점 거리 측정	셀 7: 원하는 지점 거리 측정	
8	셀 8 **(선택)**: 결과 다운로드	셀 8 **(선택)**: 결과 다운로드	셀 8 **(선택)**: 결과 다운로드	

캘리브레이션을 하지 않으면 `stereo_colab.ipynb`, `v2`, `v3`의 C1~C3 셀은 건너뛰세요. `v1`, `v2`, `v3`에는 `사용한 파일/`의 `CL.jpg`, `CR.jpg`로 실행한 결과가 저장되어 있어, 실행하지 않고도 결과를 볼 수 있습니다. 버전별 차이는 `코드_설명.md`의 5-5를 참고하세요.

정확도 높이는 팁

- 측정 가능 거리: 대략 30cm ~ 3m (baseline 6cm, VGA 기준). 멀수록 오차가 급격히 커짐
- **가까운 물체가 N/A** → `NUM_DISP`를 160, 192로 증가
- **무늬 없는 벽, 반사면, 어두운 곳**은 측정 불가 → 조명을 밝게, 질감 있는 물체로 테스트
- 좌우 밝기가 다르면 `.ino`의 `AEC_VALUE`, `AGC_GAIN`을 두 보드 동일하게 조정
- 캘리브레이션 후 카메라가 조금이라도 틀어지면 **다시 캘리브레이션** 필요
- 해상도를 바꾸면(펌웨어 `FRAMESIZE_*`) 캘리브레이션을 다시 해야 함

- 움직이는 물체는 두 카메라 촬영 시점 차이로 오차 발생 → 정지된 물체로 측정 권장
- 실측 거리와 비교해 일정 비율로 틀리면 `SQUARE_MM` 값이 정확한지 확인
- Colab 근사 측정에서 가까운 곳과 먼 곳의 오차 비율이 다르면 `DISP_OFFSET_PX`, `HFOV_DEG`를 실측 2지점으로 다시 계산 (`코드_설명.md` 7장)

3부. 코드 설명

이 문서는 `stereo` 폴더 안의 코드가 **무엇을, 왜, 어떻게** 하는지 설명합니다. 설치·실행 순서는 README.md를 보세요.

목차

1. 핵심 원리
2. 파일 구성과 전체 구조
3. ESP32 펌웨어 — esp32_stereo_cam.ino
4. PC 프로그램 — stereo.py
5. Colab 노트북 — stereo_colab.ipynb
6. 캘리브레이션 파일 — stereo_calib.npz
7. 파라미터 조정 가이드
8. 문제 해결

---

1. 핵심 원리

사람의 두 눈처럼, 같은 물체가 **왼쪽 사진과 오른쪽 사진에서 가로로 얼마나 어긋나 보이는지(시차, disparity)** 를 이용해 거리를 계산합니다.

```
          물체
          /\
         /  \        Z = 거리
        /    \
       /      \
[왼쪽 카메라]──B──[오른쪽 카메라]      B = 카메라 간격 (baseline)

      거리 Z = (초점거리 f × 카메라 간격 B) / 시차 d
```

기호	의미	단위	어디서 구하나
`f`	초점거리	픽셀	캘리브레이션 (또는 화각으로 추정)
`B`	두 카메라 사이 거리	mm	캘리브레이션 (또는 자로 측정)
`d`	좌우 사진에서 같은 점의 x좌표 차이	픽셀	StereoSGBM 알고리즘
`Z`	카메라에서 물체까지 거리	mm	위 식으로 계산

- 가까운 물체 → 시차가 큼 / 먼 물체 → 시차가 작음
- 시차 1픽셀 오차가 먼 거리에서 훨씬 큰 거리 오차가 됨 → **멀수록 부정확** (오차 $\propto$ 거리^2)

코드의 나머지는 모두 이 식을 정확하게 계산하기 위한 준비 과정입니다.

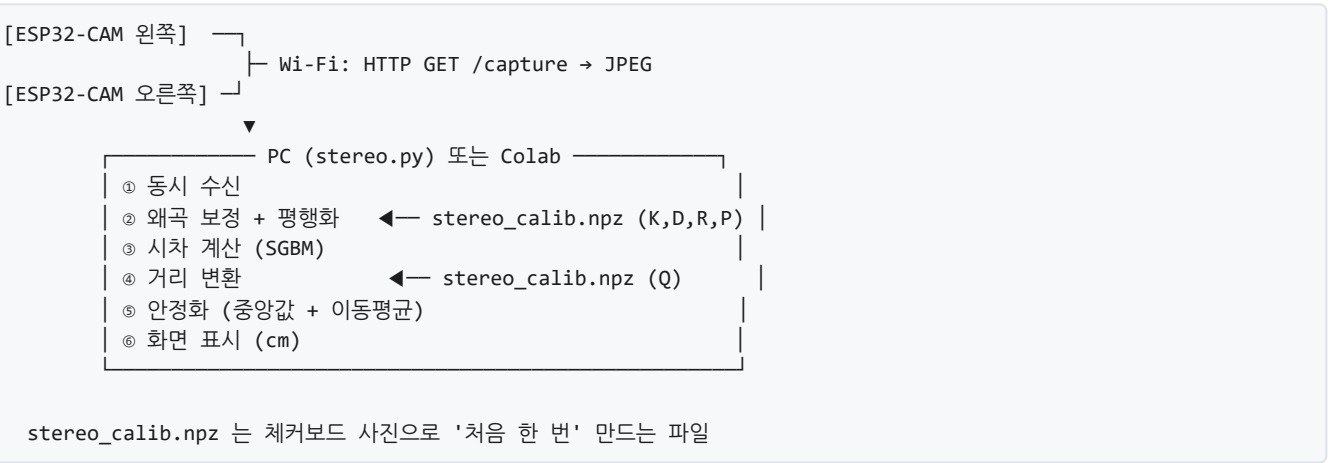
1. 두 사진을 **같은 순간에** 얻기 → 동시 촬영
2. 렌즈 왜곡 제거, 두 사진을 **같은 가로줄에 맞춤** → 캘리브레이션 + 평행화
3. 모든 픽셀의 시차 `d` 찾기 → StereoSGBM
4. `d` → 거리 `Z` 변환 → reprojectImageTo3D

---

2. 파일 구성과 전체 구조

```
stereo/
├─ esp32_stereo_cam/esp32_stereo_cam.ino   ESP32-CAM 펌웨어 (두 보드 공용)
├─ stereo.py                               PC용: 실시간 캘리브레이션 + 거리 측정
├─ stereo_colab.ipynb                      Colab용 최신본: 사진 업로드 → 캘리브레이션 / 거리 맵 (v3와 코드 같음, 기본 화각 81°)
├─ stereo_colab_v1.ipynb                   Colab v1: 사진 업로드 → 거리 맵 (캘리브레이션 셀 없음), 실행 결과 포함
├─ stereo_colab_v2.ipynb                   Colab v2: 캘리브레이션 셀 복원 + 시차 보정값(DISP_OFFSET_PX), 실행 결과 포함
├─ stereo_colab_v3.ipynb                   Colab v3: v2 + 왼쪽 가장자리까지 시차 계산, 실행 결과 포함
├─ 사용한 파일/                             측정에 사용한 이미지와 npy 파일
│   ├─ CL.jpg, CR.jpg                      사용한 왼쪽, 오른쪽 이미지 (2560×1920)
│   ├─ depth_cm (HFOV_DEG 72.0, 근사)_v1.npy  거리 맵 (640×480, cm, 화각 72°, 현재 v3 실행 결과와 같은 값)
│   └─ depth_cm(HFOV_DEG 78.0, 근사).npy     거리 맵 (640×480, cm, 화각 78°로 실행했던 이전 결과)
├─ ESP32-CAM_스테레오_거리측정_정리.pdf      진행 경과 + README + 코드 설명 + 측정 결과를 묶은 PDF
├─ requirements.txt                        PC용 파이썬 패키지 목록
├─ README.md                               설치·실행 방법
└─ 코드_설명.md                            이 문서
```

데이터 흐름



사용 방법	언제 쓰나
`stereo.py` (PC)	ESP32-CAM 두 대를 Wi-Fi로 연결해 **실시간** 측정
`stereo_colab.ipynb`	이미 찍은 **사진 2장**으로 측정, 또는 설치 없이 테스트
`stereo_colab_v1~v3.ipynb`	이전 버전과 결과를 비교할 때 (차이는 5-5 참고)

3. ESP32 펌웨어 — `esp32_stereo_cam.ino`

역할은 단 하나: `http://<IP>/capture`로 요청이 오면 사진 한 장을 찍어 JPEG로 돌려줍니다. ESP32는 CPU(240MHz)와 메모리(PSRAM 4MB)가 작아 시차 계산은 하지 않고, 계산은 모두 PC가 합니다.

3-1. 왼쪽/오른쪽 구분

```
#define CAM_SIDE 0   // 0 = 왼쪽, 1 = 오른쪽
IPAddress localIP(192, 168, 0, CAM_SIDE == 0 ? 101 : 102);
```

- 같은 코드를 두 보드에 올리고 **숫자 하나만 바꿔** 업로드합니다.

- 고정 IP를 써서 PC가 항상 같은 주소로 접속합니다.
- 공유기 대역이 `192.168.1.x` 라면 `localIP`, `gateway` 세 번째 숫자를 바꿉니다.

3-2. 촬영 처리 `handleCapture()`

```
camera_fb_t* fb = esp_camera_fb_get();
if (fb) esp_camera_fb_return(fb);   // ① 버퍼에 남아 있던 옛날 프레임 버림
fb = esp_camera_fb_get();           // ② 지금 새로 촬영
...
server.sendContent((const char*)fb->buf, fb->len);   // ③ JPEG 전송
esp_camera_fb_return(fb);           // ④ 버퍼 반납 (안 하면 다음 촬영 불가)
```

왜 첫 장을 버리나? 카메라 버퍼에는 이전에 찍어 둔 사진이 남아 있을 수 있습니다. 그대로 보내면 왼쪽은 지금, 오른쪽은 1초 전 사진이 되어 좌우 시점이 크게 달라집니다.

3-3. 카메라 설정

코드	값	이유
`frame_size`	`FRAMESIZE_VGA` (640×480)	정확도와 속도의 균형. **캘리브레이션과 측정 해상도가 같아야 함**
`jpeg_quality`	10	숫자가 작을수록 고화질 (0~63)
`fb_location`	PSRAM	VGA 이미지는 내부 RAM에 들어가지 않음
`grab_mode`	`CAMERA_GRAB_LATEST`	가장 최신 프레임 사용

3-4. 자동 노출·화이트밸런스 끄기

```
s->set_exposure_ctrl(s, 0);  s->set_aec_value(s, AEC_VALUE);  // 노출 고정
s->set_gain_ctrl(s, 0);      s->set_agc_gain(s, AGC_GAIN);    // 게인 고정
s->set_whitebal(s, 0);                                        // 화이트밸런스 고정
```

자동으로 두면 두 카메라가 **각자** 밝기를 조절해서 같은 물체가 좌우에서 다른 밝기로 찍힙니다. 시차 알고리즘은 "비슷하게 생긴 곳"을 찾는 방식이라 밝기가 다르면 매칭이 실패합니다. **두 보드의 값을 반드시 같게** 설정하세요.

3-5. `WiFi.setSleep(false)`

Wi-Fi 절전 모드에서는 응답이 수십~수백 ms 늦어질 수 있어 좌우 촬영 시점 차이가 커집니다.

---

4. PC 프로그램 — `stereo.py`

```
python stereo.py test        # 0. 연결 확인
python stereo.py collect     # 1. 캘리브레이션 사진 수집
python stereo.py calibrate   # 2. 캘리브레이션 → stereo_calib.npz
python stereo.py measure     # 3. 실시간 거리 측정
```

4-1. 설정값 (파일 맨 위)

변수	기본값	의미
`LEFT_URL`, `RIGHT_URL`	`.101`, `.102`	카메라 주소
`BOARD`	`(9, 6)`	체커보드 **내부 코너** 개수 (10×7칸 보드)

변수	기본값	의미
`SQUARE_MM`	`25.0`	체커보드 한 칸 실제 크기. **틀리면 모든 거리가 같은 비율로 틀림**
`NUM_DISP`	`128`	시차 탐색 범위(16의 배수). 가까운 물체가 안 잡히면 늘림
`BLOCK`	`5`	SGBM 비교 블록 크기(홀수)
`ROI`	`7`	거리 계산 시 주변 7×7 픽셀 중앙값 사용
`MAX_DIST_MM`	`5000`	이보다 먼 값은 버림

4-2. `grab_pair()` — 두 카메라 동시 촬영

```
fl = _pool.submit(_fetch, _sess_l, LEFT_URL)    # 스레드 1
fr = _pool.submit(_fetch, _sess_r, RIGHT_URL)   # 스레드 2
return fl.result(), fr.result()
```

- 순서대로 요청하면 한 장 받는 시간(수십~수백 ms)만큼 시점이 어긋나므로 **동시에** 요청합니다.
- `requests.Session()`은 연결을 재사용해서 매번 접속하는 시간을 줄입니다.
- 그래도 수십 ms 차이는 남으므로 **정지된 물체**에서 정확합니다.

4-3. `test()` — 연결 확인

두 영상을 나란히 띄우고 FPS를 표시합니다. **카메라 위치가 좌우로 바뀌지 않았는지** 여기서 확인합니다.

4-4. `collect()` — 캘리브레이션 사진 수집

```
okl, cl = cv2.findChessboardCorners(gl, BOARD, flags=cv2.CALIB_CB_FAST_CHECK)
```

- 실시간으로 체커보드 코너를 찾아 화면에 그립니다.
- **양쪽 모두 인식됐을 때만** SPACE로 `calib/L_00.png`, `calib/R_00.png` …가 저장됩니다.
- **체커보드를 쓰는 이유**: 칸 크기를 정확히 아는 물체라서 "실제 25mm가 화면에서 몇 픽셀인가"로 카메라 특성을 역산할 수 있습니다.

4-5. `calibrate()` — 카메라 특성 계산

단계	함수	구하는 것
① 실제 좌표	`np.mgrid * SQUARE_MM`	코너의 실제 위치 (0,0), (25,0), (50,0)… mm
② 정밀 보정	`cv2.cornerSubPix`	코너 위치를 픽셀보다 작은 단위까지
③ 카메라별	`cv2.calibrateCamera`	**K** (초점거리·중심점), **D** (렌즈 왜곡)
④ 스테레오	`cv2.stereoCalibrate`	**R** (두 카메라 사이 회전), **T** (두 카메라 사이 거리)
⑤ 평행화	`cv2.stereoRectify`	**R1, R2, P1, P2** (평행화 변환), **Q** (시차→3D 변환)

- `CALIB_FIX_INTRINSIC` : ③에서 구한 K, D를 고정하고 R, T만 계산 → 더 안정적
- `alpha=0` : 평행화 후 검은 테두리가 없도록 유효 영역만 남김
- **결과 확인법**
- RMS 오차: 0.5px 이하 우수, 1.0px 이상이면 재촬영
- 출력되는 baseline이 **자로 잰 카메라 간격과 비슷한지** 확인

평행화(rectification)란? 두 카메라가 완벽히 평행한 것처럼 이미지를 가상으로 돌려 주는 것입니다. 평행화하면 같은 물체가 **반드시 같은 가로줄(같은 y)** 에 나타나서, 매칭할 때 가로 한 줄만 찾으면 됩니다 (빠르고 정확).

4-6. `measure()` — 실시간 거리 측정

① 변환 지도 미리 계산 (시작할 때 한 번)

```
mLx, mLy = cv2.initUndistortRectifyMap(K1, D1, R1, P1, size, cv2.CV_16SC2)
```

"각 픽셀이 어디로 이동해야 하는지" 지도를 만들어 두고, 매 프레임 `cv2.remap`으로 빠르게 적용합니다.

② 시차 계산 — StereoSGBM

```
sgbm = cv2.StereoSGBM_create(minDisparity=0, numDisparities=NUM_DISP, blockSize=BLOCK,
                             P1=8*3*BLOCK**2, P2=32*3*BLOCK**2, ...)
disp = sgbm.compute(gl, gr).astype(np.float32) / 16.0
```

왼쪽 이미지 각 픽셀 주변 블록과 가장 비슷한 곳을 오른쪽 이미지 **같은 줄**에서 찾습니다.

파라미터	의미
`numDisparities`	최대 몇 픽셀 어긋남까지 찾을지. 가까울수록 시차가 크므로 넘으면 측정 불가
`blockSize`	비교 블록 크기. 크면 부드럽지만 경계가 뭉개짐, 작으면 세밀하지만 노이즈
`P1`, `P2`	이웃 픽셀끼리 시차가 급변하면 주는 벌점 → 결과가 매끄러워짐 (SGBM의 핵심)
`uniquenessRatio`	1등 후보가 2등보다 10% 이상 좋을 때만 인정 → 애매한 매칭 제거
`speckleWindowSize/Range`	작게 튀는 잡음 조각 제거
`disp12MaxDiff`	좌→우, 우→좌 결과가 1픽셀 넘게 다르면 버림
`/ 16.0`	OpenCV는 시차를 16배 정수로 돌려주므로 원래 값으로 변환

왼쪽 가장자리 처리 — `compute_disparity()` : SGBM은 왼쪽 `NUM_DISP` 픽셀을 계산하지 않으므로, 두 사진 왼쪽에 `NUM_DISP`만큼 검은 여백을 붙여 계산한 뒤 잘라냅니다. 오른쪽 카메라에 실제로 안 보이는 픽셀(`x - 시차 < 0`, 배경 시차보다 왼쪽인 열)은 제외합니다. Colab 노트북도 같은 방식입니다.

③ 시차 → 거리

```
pts = cv2.reprojectImageTo3D(disp, Q)
z = pts[:, :, 2]          # 각 픽셀까지 거리 (mm)
```

`Z = f·B/d` 계산을 `Q` 행렬로 모든 픽셀에 한꺼번에 적용합니다.

④ 믿을 수 없는 값 제거

```
valid = (disp > 0) & np.isfinite(z) & (z > 0) & (z < MAX_DIST_MM)
```

⑤ 측정 지점 거리 — `_distance_at()`

```
zs = z_map[y0:y1, x0:x1][valid[y0:y1, x0:x1]]   # 주변 7×7 중 유효한 값만
if zs.size < (ROI * ROI) // 4: return None      # 유효 값이 1/4 미만이면 측정 불가
return float(np.median(zs))
```

- 픽셀 하나만 보면 튀므로 **주변 영역의 중앙값** 사용
- 평균이 아닌 중앙값: 잘못 매칭된 극단값 몇 개에 흔들리지 않음

⑥ 흔들림 줄이기 — 지수 이동평균

```
smooth = 0.7 * smooth + 0.3 * d
```

이전 값 70% + 새 값 30% → 숫자가 프레임마다 튀지 않습니다. 지점을 새로 클릭하면 초기화.

⑦ **화면** - 왼쪽: 평행화된 영상 + 십자 + 거리(cm) + FPS - 오른쪽: 시차 컬러맵 (빨강 = 가까움, 파랑 = 멂, 검정 = 측정 불가) - 조작: 왼쪽 클릭 = 측정 지점 선택, 오른쪽 클릭 = 중앙 복귀, `s` = 스크린샷, `q` = 종료

---

5. Colab 노트북 — `stereo_colab.ipynb`

PC 설치 없이 **사진 파일**로 캘리브레이션과 거리 측정을 할 수 있습니다. 셀을 위에서부터 순서대로 실행합니다.

셀	이름	하는 일
0	준비	라이브러리 import, 그래프 한글 폰트(나눔고딕) 설치
1	설정	카메라 간격, 화각, 계산 해상도, 정렬 방식, 시차 보정값(`DISP_OFFSET_PX`) 등 입력
C1	(선택) 캘리브레이션 사진 업로드	zip 또는 왼쪽/오른쪽 사진 묶음 업로드 → 짝 맞추기
C2	(선택) 캘리브레이션 실행	코너 인식 → 캘리브레이션 → 나쁜 사진 제외 → `stereo_calib.npz` 저장
C3	(선택) 확인 + 다운로드	평행화 결과 확인, npz 파일 다운로드
2	측정할 사진 업로드	왼쪽, 오른쪽 사진 (+ npz 파일, C2를 했으면 자동 사용)
3	정렬	캘리브레이션 평행화 또는 세로 정렬(y-align)
4	시차 계산	탐색 범위 자동 설정, 좌우 바뀜 자동 감지, 왼쪽 가장자리 여백 처리, SGBM (+ WLS 필터)
5	거리 계산	시차에 `DISP_OFFSET_PX` 를 더한 뒤 → cm
6	결과 보기	시차/거리 맵, 마우스를 올리면 거리가 보이는 그림
7	지점 측정	x, y 좌표 입력 → 거리
8	다운로드	`depth_cm.npy` , `depth_color.png`

5-1. 캘리브레이션 셀 (C1~C3)

사진 짝 맞추기 (C1) - zip: 파일 이름에서 `L_` ↔ `R_` , `left` ↔ `right` 부분을 지운 나머지가 같으면 한 쌍 (`L_03.png` ↔ `R_03.png`) - 직접 업로드: 왼쪽 묶음, 오른쪽 묶음을 각각 **이름 순(자연 정렬: IMG_2 < IMG_10)** 으로 정렬해 순서대로 짝

코너 인식 (C2 `find_corners`)

```
ok, c = cv2.findChessboardCornersSB(gray, BOARD, ...)   # 1차: 정확도 높은 최신 알고리즘
if not ok:
    ok, c = cv2.findChessboardCorners(gray, BOARD, ...) # 2차: 기존 알고리즘
    c = cv2.cornerSubPix(...)                           #      + 서브픽셀 보정
```

좌우 코너 순서 확인

```
if np.dot((ca[-1] - ca[0]).ravel(), (cb[-1] - cb[0]).ravel()) < 0:
    cb = cb[::-1].copy()
```

체커보드가 기울어 있으면 왼쪽은 왼쪽 위부터, 오른쪽은 오른쪽 아래부터 번호를 매길 수 있습니다. 첫 코너→마지막 코너 방향이 좌우에서 반대면 순서를 뒤집어 맞춥니다.

나쁜 사진 자동 제외

```
bad = res["per"] > max(1.0, 2.5 * np.median(res["per"]))
```

사진별 재투영 오차를 계산해 **1px 이상이면서 중앙값의 2.5배를 넘는** 사진(흔들림, 오인식)을 빼고 다시 계산합니다.

결과 출력: RMS 오차, 측정된 카메라 간격, 초점거리·화각. 측정된 간격이 설정값과 20% 이상 다르면 경고합니다.

해상도 `CALIB_WIDTH` : 캘리브레이션 사진을 이 가로 크기로 줄여서 계산합니다. 측정 사진이 더 커도 **가로세로 비율이 같으면 자동으로 같은 크기로 맞춰** 캘리브레이션을 그대로 씁니다 (스마트폰 원본 사진 등).

5-2. 정렬 방식 (셀 3)

모드	조건	방법	정확도
`calib`	npz 파일 있음	왜곡 보정 + 평행화 (PC와 동일)	높음
`y-align`	npz 없음 (기본)	특징점으로 **세로 어긋남만** 보정	근사
`none`	직접 선택	보정 안 함	카메라가 완벽히 평행할 때만

y-align의 원리

```
pl, pr = match_points(L, R)              # SIFT 특징점 매칭
dy = pl[:, 1] - pr[:, 1]                 # 좌우 세로 위치 차이
# dy = a + b·x + c·y 를 최소제곱으로 구함 (이상치 제거하며 5번 반복)
map_y = (ys - a - b*xs) / (1 + c)        # 오른쪽 사진의 '세로'만 이동
R = cv2.remap(R, xs, map_y, ...)
```

- `a` : 위아래 평행 이동, `b` : 살짝 기울어짐(roll), `c` : 세로 크기 차이
- **가로(x)는 건드리지 않습니다.** 가로까지 바꾸면 "아주 먼 곳 = 시차 0" 관계가 깨져 `Z = f·B/d` 공식을 쓸 수 없기 때문입니다.

초점거리 추정 (npz가 없을 때)

```
f_px = (w / 2) / tan(HFOV / 2)
Q = [[1, 0, 0, -w/2], [0, 1, 0, -h/2], [0, 0, 0, f_px], [0, 0, 1/B, 0]]
```

화각과 이미지 폭으로 초점거리를 계산하고, PC 버전과 같은 형식의 `Q` 행렬을 직접 만듭니다. 렌즈 왜곡은 보정하지 못하므로 **근사값**입니다.

5-3. 시차 탐색 범위 자동 설정 (셀 4)

```
d_pts = pl[:, 0] - pr[:, 0]                       # 특징점들의 실제 시차
lo, hi = np.percentile(d_pts, [1, 99.5])
min_disp = max(0, floor(lo) - 8)
num_disp = ceil((hi * 1.5 + 16 - min_disp) / 16) * 16
```

- 사진마다 거리가 달라 고정값(128)은 너무 좁거나 넓을 수 있으므로, **특징점 시차의 분포**로 범위를 정합니다.
- 위쪽은 1.5배 + 16px 여유: 특징점이 없는 더 가까운 물체도 잡기 위해
- 시차 중앙값이 음수면 **좌우 사진이 바뀐 것**으로 판단해 자동 교체

왼쪽 가장자리 여백 처리: OpenCV SGBM은 왼쪽 `min_disp + num_disp` 픽셀을 계산하지 않습니다(짝을 찾을 오른쪽 사진 범위가 화면 밖으로 나가기 때문). 그래서 두 사진 왼쪽에 그만큼 검은 여백을 붙여 계산한 뒤 잘라냅니다. 단, 짝이 오른쪽 사진 밖에 있는 픽셀(`x - 시차 < 0`)과 가장 먼 배경의 시차보다 왼쪽에 있는 열은 **오른쪽 카메라에 아예 안 보이는 영역**이므로 제외합니다.

WLS 필터 (Colab에 `opencv-contrib`가 있으면 자동 적용): 좌→우, 우→좌 시차를 모두 계산한 뒤 원본 이미지의 경계를 참고해 빈 곳을 메우고 경계를 다듬습니다.

5-4. 결과 보기 (셀 6)

- matplotlib: 왼쪽 사진 / 시차 맵 / 거리 맵(cm) 3장
- plotly: 사진 위에 거리 맵을 반투명하게 겹치고, **마우스를 올리면 좌표와 거리(cm) 표시**. 큰 이미지는 가로 400픽셀 수준으로 줄여 브라우저 부담을 줄임

5-5. 노트북 버전별 차이

`stereo_colab.ipynb`는 최신본으로, 코드는 `stereo_colab_v3.ipynb`와 같고 기본 화각(`HFOV_DEG`)만 81°로 다릅니다. 위 셀 설명은 이 최신본 기준입니다.

항목	`v1`	`v2`	`v3` (코드는 `stereo_colab.ipynb`와 같음)
셀 구성	1~8 (캘리브레이션 셀 없음)	0, 1, C1~C3, 2~8	v2와 같음
캘리브레이션	`python stereo.py calibrate`로 만든 npz를 셀 2에서 업로드	C1~C3 셀에서 직접 생성 가능	v2와 같음
사진과 npz의 크기가 다를 때	npz를 쓰지 않고 y-align으로 진행	비율이 같으면 캘리브레이션 크기로 맞춤	v2와 같음
시차 보정값 `DISP_OFFSET_PX`	없음	있음 (7장의 식으로 계산)	v2와 같음
왼쪽 가장자리	왼쪽 `min_disp + num_disp` 픽셀은 측정 불가	v1과 같음	여백을 붙여 계산 (5-3)
WLS 필터를 쓸 수 없을 때	함수 지원 여부도 확인하고, 실패하면 안내 후 기본 시차 맵 사용	`cv2.ximgproc`가 없으면 건너뜀	v2와 같음
기본 설정값	간격 50mm, 화각 72°, `BLOCK_SIZE` 9, `MAX_DIST_CM` 200, `ROI` 31	간격 50mm, 화각 72°, `BLOCK_SIZE` 5, `MAX_DIST_CM` 500, `ROI` 9, `DISP_OFFSET_PX` 4.1	v2와 같음
저장된 실행 결과	있음	있음	있음

`stereo_colab.ipynb`는 기본 화각만 81°이고 나머지 설정은 v3와 같습니다.

v1, v2, v3에 저장된 실행 결과 (`사용한 파일/`의 `CL.jpg`, `CR.jpg`, 화각 72°, 캘리브레이션 없는 근사 측정)

항목	`v1`	`v2`	`v3`
초점거리 추정	약 440px	v1과 같음	v1과 같음
세로 정렬	매칭점 440/505개, 세로 어긋남 5.8px → 0.41px	v1과 같음	v1과 같음
시차 탐색 범위	21 ~ 101 px	v1과 같음	v1과 같음
유효 픽셀 비율	84%	84%	95% (측정 가능한 가로 범위 30 ~ 639 px)
측정된 거리 범위	42.4 ~ 104.6 cm	38.0 ~ 87.5 cm	38.0 ~ 87.5 cm
화면 중앙 거리	57.2 cm	51.5 cm	51.5 cm

- v2, v3의 거리가 v1보다 짧게 나오는 것은 시차 보정값 `DISP_OFFSET_PX` (4.1px)를 더하기 때문입니다.
- v3는 왼쪽 가장자리 여백 처리 덕분에 유효 픽셀이 늘었습니다.
- 셀 8에서 받은 거리 맵은 `사용한 파일/`에 화각별로 저장되어 있습니다.
- `depth_cm (HFOV_DEG 72.0, 근사)_v1.npy`: 38.0 ~ 87.5 cm, 유효 픽셀 95%. 위 v3 결과와 같은 값입니다.
- `depth_cm(HFOV_DEG 78.0, 근사).npy`: 34.1 ~ 78.5 cm. 화각 78°로 실행했던 이전 결과입니다.
- 모두 실측과 비교한 값이 아니므로 동작 예시로만 보세요.

---

6. 캘리브레이션 파일 — `stereo_calib.npz`

`stereo.py calibrate` 또는 Colab C2 셀이 만드는 파일입니다. **두 프로그램이 같은 형식**이라 서로 바꿔 쓸 수 있습니다.

키	크기	의미	사용처
`K1`, `K2`	3×3	카메라 행렬 (초점거리 fx, fy / 중심 cx, cy)	왜곡 보정
`D1`, `D2`	1×5	렌즈 왜곡 계수 (k1, k2, p1, p2, k3)	왜곡 보정
`R1`, `R2`	3×3	평행화 회전	평행화
`P1`, `P2`	3×4	평행화 후 투영 행렬	평행화
`Q`	4×4	시차 → 3D 좌표 변환	거리 계산
`size`	2	캘리브레이션 해상도 (가로, 세로)	해상도 확인

⚠ **카메라 간격·각도·해상도가 바뀌면 다시 만들어야 합니다.** 다른 사람이 만든 파일은 쓸 수 없습니다 (렌즈마다 값이 다름).

---

7. 파라미터 조정 가이드

증상	조정할 값	방향
가까운 물체가 측정 안 됨	`NUM_DISP` (stereo.py)	128 → 160, 192
시차 맵에 구멍이 많음	`BLOCK` / `BLOCK_SIZE`	5 → 7, 9
물체 경계가 뭉개짐	`BLOCK` / `BLOCK_SIZE`	5 → 3
거리 숫자가 많이 흔들림	`ROI`, 이동평균 비율	ROI 7 → 11, `0.7/0.3` → `0.85/0.15`
거리가 일정 비율로 틀림	`SQUARE_MM` (캘리브레이션) / `HFOV_DEG`, `BASELINE_MM` (근사)	실측값으로 수정
가까운 곳과 먼 곳의 오차 비율이 다름	`DISP_OFFSET_PX` + `HFOV_DEG` (Colab)	실측 2지점으로 계산 (아래 식)
화면이 어둡거나 밝음	`AEC_VALUE`, `AGC_GAIN` (펌웨어)	두 보드 같은 값으로
더 먼 거리를 재고 싶음	카메라 간격	넓히고 다시 캘리브레이션

실측 2지점으로 `DISP_OFFSET_PX`, `HFOV_DEG` 구하기 (Colab 근사 모드)

카메라가 완벽히 평행하지 않으면 시차에 일정한 오차 `d0`가 더해져, 단순 배율로는 가까운 곳과 먼 곳을 동시에 맞출 수 없습니다. 그래서 `Z = K / (d + d0)` 모델을 씁니다.

```
현재 설정: K_old = BASELINE_MM × (320 / tan(HFOV_DEG/2))      (가로 640px 기준)
측정 시차: d1 = K_old / 측정거리1(mm),  d2 = K_old / 측정거리2(mm)
K  = (d1 - d2) / (1/실제거리1 - 1/실제거리2)
d0 = K / 실제거리1 - d1                      → DISP_OFFSET_PX
f  = K / BASELINE_MM,  HFOV_DEG = 2 × atan(320 / f)
```

예) 50mm / 78°에서 실제 35cm→40cm, 65cm→80cm로 측정됨 → `DISP_OFFSET_PX = 4.1`, `HFOV_DEG = 81`

측정 가능 거리 예시 (간격 6cm, VGA, 초점거리 ≈ 490px)

거리	시차	시차 1px 오차 시 거리 오차
50cm	59px	±0.9cm
1m	29px	±3.4cm
2m	15px	±14cm
3m	10px	±30cm

---

8. 문제 해결

증상	원인	해결
`test`에서 연결 안 됨	IP / Wi-Fi 설정	브라우저로 `http://192.168.0.101/capture` 접속 확인, PC와 같은 공유기인지 확인 (ESP32는 2.4GHz만 지원)
체커보드 인식 실패	`BOARD` 값, 조명	**내부 코너 수** 확인 (칸 수 − 1), 반사·그림자 없게, 보드 전체가 보이게
RMS 오차 1px 이상	흔들린 사진, 보드가 휨	판에 평평하게 붙이고 다시 촬영, 다양한 각도·위치로 20장 이상
측정된 baseline이 실제와 다름	`SQUARE_MM` 오류	칸 크기를 자로 다시 재기 (프린터 배율 확인)
시차 맵이 거의 비어 있음	좌우 바뀜, 정렬 불량, 무늬 없음	좌우 확인, 평행화 그림에서 가로줄 확인, 무늬 있는 장면으로
거리가 측정 불가 (N/A)	시차 범위 초과, 매칭 실패	`NUM_DISP` 늘리기, 무늬 있는 지점 선택
움직이는 물체 거리가 이상함	좌우 촬영 시점 차이	정지된 물체로 측정 (정밀하게 하려면 GPIO 트리거로 동기화)
Colab 그래프 한글이 □로 나옴	폰트 설치 실패	0번 셀 다시 실행 또는 런타임 다시 시작

4부. 측정 예시 결과

`사용한 파일/` 의 이미지를 `stereo_colab_v3.ipynb` 로 실행한 결과입니다. 캘리브레이션 없이 화각(72°)과 카메라 간격(50mm)으로 초점거리를 추정한 **근사 측정**이며, 실측과 비교한 값은 아닙니다.

- `CL.jpg` , `CR.jpg` : 2560×1920 원본, 계산 시 가로 640px로 축소
- `depth_cm (HFOV_DEG 72.0, 근사)_v1.npy` : 640×480 거리 맵, 유효 범위 38.0 ~ 87.5 cm (아래 출력과 같은 값)
- `depth_cm(HFOV_DEG 78.0, 근사).npy` : 640×480 거리 맵, 유효 범위 34.1 ~ 78.5 cm (화각 78°로 실행했던 이전 결과)



CL.jpg (왼쪽)



CR.jpg (오른쪽)

v3 노트북 셀별 출력

0. 준비 (라이브러리, 한글 폰트)

```
준비 완료, OpenCV 4.14.0
```

C1. 캘리브레이션 사진 업로드

```
zip 파일이 있으면 zip을, 없으면 '왼쪽 사진들'을 모두 선택하세요
```

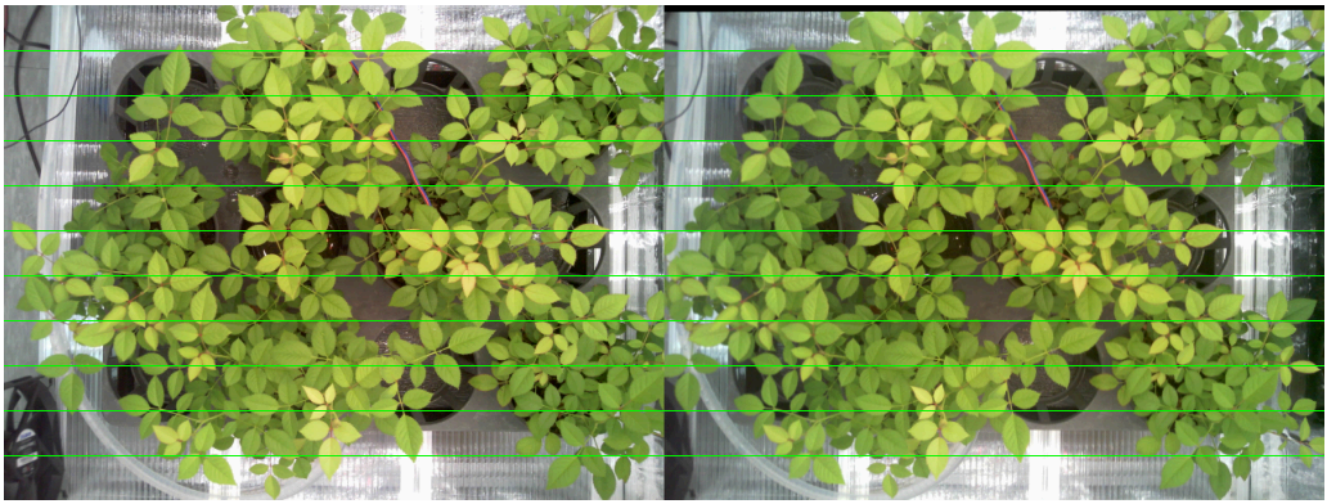
2. 측정할 사진 업로드

```
① 왼쪽 사진을 선택하세요
Saving CL.jpg to CL.jpg
② 오른쪽 사진을 선택하세요
Saving CR.jpg to CR.jpg
③ (선택) stereo_calib.npz 를 올리려면 선택, 없으면 '취소'를 누르세요
왼쪽: CL.jpg
오른쪽: CR.jpg
캘리브레이션: 없음 (근사 측정)
```

3. 사진 불러오기 + 정렬(평행화)

```
✓ 세로 정렬 완료: 매칭점 440/505개 사용, 보정 전 세로 어긋남 중앙값 5.8px → 보정 후 0.41px
근사 측정 모드: 초점거리 ≈ 440px (화각 72.0°), 카메라 간격 50.0mm
```

정렬 확인: 같은 물체가 초록 가로줄 기준으로 같은 높이에 있어야 함 (왼쪽 | 오른쪽)



4. 시차(disparity) 계산

```
시차 탐색 범위: 21 ~ 101 px
✓ WLS 필터 적용
유효 픽셀 비율: 95%
측정 가능한 가로 범위: 30 ~ 639 px (왼쪽 30px는 오른쪽 카메라에 안 보이는 영역)
```

5. 거리 계산

```
측정된 거리 범위: 38.0 ~ 87.5 cm
화면 중앙 거리: 51.5 cm
```

7. 원하는 지점 거리 측정



5부. 업로드 기록

커밋	내용
`44866be`	프로젝트 첫 업로드 (펌웨어, `stereo.py`, `stereo_colab.ipynb`, README, 코드 설명)
`4fa7a61`	`stereo_colab_v1.ipynb` 추가 (출력 제거), Colab 실행 순서 문서화
`d7e4cda`	README 제목에 날짜 추가
`b5b3b00`	`stereo_colab_v2.ipynb`, `stereo_colab_v3.ipynb`, 사용한 파일/ 추가, 문서 갱신
`c4f36b1`	README의 Colab 실행 순서에 v2, v3 열 추가

이 문서를 만들기 전까지의 기록입니다. 이 PDF는 그다음 업로드에 함께 올라갑니다.

저장소: `https://github.com/Lumiela/helpai/tree/Height` (`main` 브랜치는 변경하지 않음)